

자바 함수형, 스트림, MySQL 설치

오늘의 강의 학습 요약

오늘은 자바의 표준 함수형 인터페이스(Consumer, Supplier, Function, Operator, Predicate)를 타입 관점에서 정리하고, BinaryOperator 실습으로 "입력 타입과 반환 타입의 관계"를 확실히 구분했습니다. 이어서 Predicate의 논리 결합(and, or, negate)과 Function/Consumer의 작업 연결(andThen)을 통해 람다 기반 파이프라인 사고를 확장했으며, 람다를 더 축약하는 메소드 참조(정적/인스턴스/생성자/System.out::println)까지 실습으로 정리했습니다.

후반부는 Stream API로 넘어가 컬렉션/배열에서 스트림을 얻는 방법부터 시작해, 중간 처리(중복 제거, 필터, 맵, 플랫맵, 정렬, 스킵, 리밋)와 최종 처리(카운트, 최대/최소, 평균, 합계, 수집 변환)를 단계적으로 연결했습니다. 특히 스트림의 일회성(terminal 이후 재사용 불가), IntStream 등 기본형 스트림의 추가 메소드, collect(Collectors...) 구조, 그리고 Optional의 기본값 처리(orElse)를 강조했습니다. 마지막으로 내일 DB 수업을 위한 MySQL 8 설치 및 Workbench 연결 설정을 진행했습니다.

1. 표준 함수형 인터페이스 정리

`Consumer`, `Supplier`, `Function`, `Operator`, `Predicate`는 “파라미터 유무/개수”와 “리턴 타입 유무/형태”로 구분되며, `Operator`는 입력과 출력 타입이 동일하다는 점이 `Function`과의 핵심 차이입니다.

`java.util.function` 패키지에는 자주 쓰는 함수형 인터페이스가 목적별로 정의되어 있습니다. 가장 중요한 구분 기준은 “입력(파라미터)이 있는가”와 “출력(리턴)이 있는가”입니다.

인터페이스 계열	입력(파라미터)	출력(리턴)	핵심 용도
<code>Consumer</code>	있음	없음	값을 소비하며 출력/저장/로그 등 부수효과를 수행합니다.
<code>Supplier</code>	없음	있음	값을 공급하며 지연 생성, 기본값 생성 등에 사용됩니다.
<code>Function</code>	있음	있음	입력을 다른 값으로 변환(매핑)합니다.
<code>Operator</code>	있음	있음	입력과 출력의 타입이 동일한 변환/결합에 적합합니다.
<code>Predicate</code>	있음	<code>boolean</code>	조건 판별(필터링)에 사용됩니다.

`Consumer` 계열은 반환값이 없고 입력만 받습니다. 입력 개수는 `Consumer`, `BiConsumer` 처럼 1개 또는 2개 형태가 대표적이며, 보통 출력이나 상태 변경 같은 작업에 사용됩니다.

`Supplier` 계열은 입력이 없고 결과를 반환합니다. 스트림 API에서 값 생성(예: 초기값 공급, 지연 계산) 패턴으로 활용되는 경우가 많습니다.

`Function` 은 입력과 출력이 모두 존재하는 범용 변환 인터페이스입니다. 입력 타입과 출력 타입이 서로 달라도 되므로, “문자열 → 길이(정수)” 같은 변환을 자연스럽게 표현할 수 있습니다.

`Operator` 는 `Function` 의 특수 형태로, 입력과 출력 타입이 동일합니다. 따라서 “같은 타입을 받아 같은 타입을 반환하는 계산/조합”에 가장 적합합니다.

`Predicate` 는 입력을 받아 `boolean` 을 반환하는 조건 인터페이스입니다. 필터링처럼 “참인 것만 남기는” 처리에 사용됩니다.

핵심 키워드

`#Consumer`

`#Supplier`

`#Function`

`#Operator`

`#Predicate`

2. Operator 실습: BinaryOperator

`BinaryOperator<T>` 는 입력 2개와 반환 1개의 타입이 모두 동일한 연산을 표현하며, `apply(T, T)` 로 두 값을 결합한 결과를 반환합니다.

`BinaryOperator<T>` 는 “두 입력을 받아 하나의 결과를 반환”하는 형태이며, `Operator` 특성상 입력/출력 타입이 모두 `T` 로 동일합니다. 예를 들어 문자열 두 개를 받아 결합한 문자열을 반환하는 연산에 적합합니다.

`BinaryOperator` 의 개념을 코드 형태로 보면 `apply(T t1, T t2) -> T` 구조로 이해하시면 됩니다. 문자열 결합처럼 “두 값을 합쳐 같은 타입으로 돌려주는” 패턴이 대표적입니다.

다음과 같은 흐름으로 실습이 구성됩니다.

- `BinaryOperator<String>` 을 만들면 입력 2개와 반환 1개가 모두 `String` 으로 고정됩니다.
- 익명 클래스 형태로 먼저 구현한 뒤, 동일 동작을 람다 표현식으로 축약할 수 있습니다.
- `apply` 호출 시 전달한 두 문자열이 결합되어 반환됩니다.

핵심 키워드

#BinaryOperator

#apply

#제네릭

#익명 클래스

#람다

3. Predicate: 논리 결합(and/or/negate)

`Predicate<T>` 는 `test(T)` 가 `boolean` 을 반환하며, `and` , `or` , `negate` 로 조건을 조합하거나 부정할 수 있습니다.

`Predicate<T>` 는 어떤 값이 조건을 만족하는지 판별하는 데 사용됩니다. 기본 메소드는 `test(T t)` 이고, 반환 타입은 항상 `boolean` 입니다. 예를 들어 문자열 길이가 3인지 여부를 검사하는 조건을 만들 수 있습니다.

또한 `BiPredicate<T, U>` 는 파라미터를 2개 받아 `boolean` 을 반환합니다. 문자열과 인덱스를 받아 특정 위치 문자가 특정 값인지 판별하는 형태처럼 "두 입력 기반의 조건"을 표현할 수 있습니다.

`Predicate` 계열은 논리 결합 메소드가 내장되어 조건식을 조립할 수 있습니다.

메소드	의미	예시 의도
<code>and</code>	두 조건이 모두 참일 때 참입니다.	2의 배수이면서 3의 배수인지 검사합니다.
<code>or</code>	둘 중 하나라도 참이면 참입니다.	2의 배수이거나 3의 배수인지 검사합니다.
<code>negate</code>	조건을 부정합니다.	2의 배수가 아닌지 검사합니다.

정수 조건 예제에서는 `IntPredicate` 를 사용해 2의 배수/3의 배수 조건을 만든 후 `and` , `or` , `negate` 로 결합해 테스트했습니다. 이때 핵심은 "논리 연산을 메소드로 표현한다"는 점이며, 스트림의 `filter` 와 직접 연결됩니다.

핵심 키워드

`#Predicate`

`#BiPredicate`

`#and`

`#or`

`#negate`

4. andThen: 작업 연결(Consumer/Function)

`andThen` 은 `Consumer` 또는 `Function` 에서만 제공되며, 앞 작업의 결과(또는 동일 입력)를 다음 작업으로 연결해 파이프라인을 구성합니다.

`andThen` 은 “작업을 하나로 끝내지 않고 이어서 수행”하기 위한 조합 메소드입니다. 단, 모든 함수형 인터페이스에 있는 것은 아니며, 강의에서는 `Function` 과 `Consumer` 에만 있음을 확인했습니다.

`Consumer.andThen` 은 같은 입력을 두 `Consumer`에 순서대로 전달합니다. 즉, 첫 `Consumer`가 입력을 소비하고, 이어서 두 번째 `Consumer`도 동일 입력을 소비합니다. 결과적으로 두 작업을 묶은 “새 `Consumer`”가 만들어집니다.

`Function.andThen` 은 첫 `Function`의 반환값이 두 번째 `Function`의 입력으로 들어가므로, 타입 정합성이 매우 중요합니다. 예를 들어 `String -> Integer` 변환을 먼저 하고, 이어서 `Integer -> Integer` 처리를 연결하는 형태가 가능합니다.

`Function.andThen` 연결 시 중요한 조건은 다음과 같습니다.

- 첫 번째 함수의 반환 타입과 두 번째 함수의 입력 타입이 반드시 일치해야 합니다.
- 일치하지 않으면 컴파일 오류가 발생합니다.
- `Operator` 는 `Function` 의 하위 계열이므로 `andThen` 을 상속받아 사용할 수 있습니다.

핵심 키워드

#andThen

#Consumer

#Function

#타입정합성

#체이닝

5. 메소드 참조(Method Reference)

메소드 참조는 람다를 더 축약한 문법이며, 정적 메소드, 인스턴스 메소드, 특정 타입의 인스턴스 메소드, 생성자, `System.out::println` 형태로 표현됩니다.

메소드 참조는 “람다로 구현한 내용이 단순히 기존 메소드 호출로 대응될 때” 간결하게 표현하는 문법입니다. 콜론 두 개(`::`)를 사용합니다.

메소드 참조는 대표적으로 다음 네 형태가 사용됩니다.

형태	문법	의미
정적 메소드	<code>클래스명::staticMethod</code>	클래스의 정적 메소드를 참조합니다.
인스턴스 메소드(객체 보유)	<code>참조변수::method</code>	특정 객체의 인스턴스 메소드를 참조합니다.
특정 타입의 인스턴스 메소드	<code>타입명::method</code>	입력 객체의 인스턴스 메소드를 호출하는 형태로 해석됩니다.
생성자 참조	<code>클래스명::new</code>	생성자 호출을 참조합니다.

예를 들어 `Integer.parseInt` 는 `String -> int` 변환이므로 `Function<String, Integer>` 또는 기본형 특화 함수형 인터페이스와 함께 `Integer::parseInt` 로 축약할 수 있습니다.

또한 출력은 `Consumer<T>` 로 자주 쓰이므로 `System.out.println(x)` 형태는 `System.out::println` 로 축약하는 패턴이 실무에서 매우 빈번합니다.

핵심 키워드

#메소드참조

#클래스명::메소드

#참조변수::메소드

#클래스명::new

#System.out::println

6. Stream API 개요와 생성(컬렉션/배열)

스트림은 컬렉션, 배열의 대량 데이터를 “흘러보내며” 중간 처리와 최종 처리를 수행하는 API이며, 사용 전 반드시 스트림을 먼저 생성해야 합니다.

스트림은 “데이터를 순회하면서 필터링, 정렬, 가공, 통계 처리” 같은 작업을 선언적으로 수행하기 위한 도구입니다. 처리 흐름은 일반적으로 “스트림 생성 → 중간 처리(0회 이상) → 최종 처리(1회)”로 구성됩니다.

스트림 생성은 데이터 소스에 따라 방식이 달라집니다.

데이터 소스	스트림 얻는 방법	비고
List, Set 등 Collection	<code>collection.stream()</code>	<code>stream()</code> 은 <code>Collection</code> 에 정의되어 있어 하위에서 공통 사용됩니다.
배열	<code>Arrays.stream(array)</code>	배열은 메소드가 없으므로 <code>Arrays</code> 유틸을 사용합니다.
배열(대안)	<code>Stream.of(array)</code>	배열을 스트림으로 만드는 또 다른 방법입니다.

스트림 객체는 다양한 중간/최종 처리 메소드를 제공하며, `distinct`, `filter`, `map`, `flatMap`, `sorted`, `skip`, `limit` 같은 중간 처리와 `forEach`, `count`, `collect` 같은 최종 처리를 결합합니다.

핵심 키워드 `#Stream` `#Collection.stream` `#Arrays.stream` `#Stream.of` `#중간처리`

7. forEach와 스트림 메소드의 함수형 인터페이스

`forEach`의 파라미터는 `Consumer`이며, 스트림의 많은 메소드가 내부 파라미터로 함수형 인터페이스를 받기 때문에 람다, 메소드 참조가 핵심 문법이 됩니다.

스트림에서 반복 출력은 `forEach`로 수행하며, `forEach`는 내부적으로 `Consumer`를 전달받습니다. 따라서 “데이터를 하나씩 소비하며 출력”하는 작업은 `System.out::println` 같은 메소드 참조로 자연스럽게 표현됩니다.

실습에서는 `Consumer`를 익명 클래스 → 람다 → 메소드 참조 순서로 축약해, 최종적으로는 다음과 같은 형태가 실무 코드의 기본형임을 강조했습니다.

```
list.stream().forEach(System.out::println);
```

이 흐름을 이해하면, 스트림의 다른 메소드(`filter`는 `Predicate`, `map`은 `Function`)도 동일한 사고방식으로 연결할 수 있습니다.

핵심 키워드

#forEach

#Consumer

#람다

#메소드참조

#메소드체인

8. 중간 처리: distinct, filter, map, flatMap

중간 처리는 항상 스트림을 반환해 체이닝이 가능하며, `distinct` 는 중복 제거, `filter` 는 조건 추출, `map` 은 1:1 변환, `flatMap` 은 1:N 펼침을 수행합니다.

`distinct()` 는 중복 요소를 제거하는 중간 처리입니다. 반환 타입이 스트림이므로 이후에 계속 다른 중간 처리나 최종 처리를 연결할 수 있습니다.

다만 스트림은 일회성이라서, `forEach` 같은 최종 처리를 수행한 뒤 동일 스트림을 다시 사용하면 오류가 발생합니다. 따라서 다시 처리하려면 원본 컬렉션에서 스트림을 다시 생성해야 합니다.

`filter(predicate)` 는 `Predicate` 를 받아 조건이 `true` 인 요소만 통과시킵니다. 예를 들어 "이름 길이가 3인 것만 출력" 같은 조건을 람다로 구성할 수 있습니다.

`map(function)` 은 요소를 다른 형태로 가공하는 메소드이며, 예제에서는 `Student` 객체에서 `getName()` 만 뽑아 `Stream<String>` 을 만드는 형태를 실습했습니다. 이때 `Function<Student, String>` 을 익명 클래스/람다/메소드 참조(`Student::getName`)로 축약할 수 있습니다.

`flatMap` 은 하나의 입력이 "여러 개의 결과(스트림)"로 확장되는 경우에 사용됩니다. 실습에서는 쉼표로 구분된 문자열을 `split` 으로 쪼갬 뒤, 정수 배열로 변환하고 `IntStream` 으로 펼쳐 반복 처리할 수 있음을 확인했습니다.

메소드	입력 → 출력 개념	대표 예
<code>map</code>	1개 → 1개	<code>Student</code> → <code>name</code> 변환입니다.
<code>flatMap</code>	1개 → 여러 개	<code>"10,20,30"</code> → <code>10,20,30</code> 으로 펼침입니다.

핵심 키워드

`#distinct`

`#filter`

`#map`

`#flatMap`

`#스트림일회성`

9. 중간 처리: sorted, skip, limit

`sorted()` 는 기본 오름차순 정렬이며, `sorted(Comparator)` 로 정렬 기준을 지정할 수 있고, `skip` , `limit` 로 일부 구간을 잘라낼 수 있습니다.

정렬은 `sorted()` 또는 `sorted(Comparator)` 로 수행합니다. 숫자처럼 기본 비교가 가능한 타입은 `sorted()` 만으로 오름차순 정렬이 됩니다.

내림차순은 `Comparator.reverseOrder()` 를 `sorted` 에 전달해 구현할 수 있습니다. 이때 `reverseOrder` 는 정적 메소드이므로 `Comparator.reverseOrder()` 형태로 호출합니다.

객체 정렬은 "어떤 필드로 비교할지"를 알려줘야 하며, `Comparator.comparing(function)` 을 사용합니다. 여기서 `function` 은 객체를 받아 정렬 키(예: 점수)를 반환하는 함수이므로, `Student -> grade` 형태의 함수가 들어갑니다.

`skip(n)` 은 앞에서 `n` 개를 건너뛰고, `limit(n)` 은 최대 `n` 개까지만 통과시키는 중간 처리입니다. 둘 다 스트림을 반환하므로 다른 처리와 체이닝이 가능합니다.

핵심 키워드

`#sorted`

`#Comparator.comparing`

`#reverseOrder`

`#skip`

`#limit`

10. 최종 처리: count, max/min, IntStream 확장

최종 처리는 스트림이 아닌 값을 반환하며, 배열 기반 `IntStream` 은 `sum` , `average` 등 기본형 특화 통계 메소드를 추가로 제공합니다.

`count()` 는 요소 개수를 `long` 으로 반환하는 대표적인 최종 처리입니다. 중간 처리(`filter` 등)를 먼저 적용한 뒤 `count()` 를 호출하면 조건을 만족하는 개수만 계산할 수 있습니다.

최댓값/최솟값은 `max` , `min` 으로 구할 수 있으나 반환이 `Optional` 계열로 내려오는 경우가 많습니다. 값이 없을 수 있는 상황(빈 스트림)을 안전하게 다루기 위한 설계입니다.

배열에서 `Arrays.stream(int[])` 로 스트림을 얻으면 `Stream<Integer>` 가 아니라 `IntStream` 이 생성됩니다. `IntStream` 은 `Stream` 보다 통계 메소드가 풍부하며, `sum()` , `average()` , 비교자 없는 `max()/min()` 등이 제공됩니다.

구분	예시 생성	대표 통계 메소드
<code>Stream<Integer></code>	<code>list.stream()</code>	범용 메소드 위주로 제공됩니다.
<code>IntStream</code>	<code>Arrays.stream(intArray)</code>	<code>sum</code> , <code>average</code> 등 숫자 특화 메소드가 추가됩니다.

핵심 키워드

`#count`

`#max`

`#min`

`#IntStream`

`#average`

11. collect와 Collectors: 리스트/셋 변환 및 통계

`collect` 는 `Collector` 를 받아 결과 컨테이너로 수집하며, `Collector` 는 `Collectors` 유틸에서 생성해 `toList` , `toSet` , `counting` , `summingInt` 등을 사용합니다.

스트림 결과를 `List` 나 `Set` 으로 바꾸려면 `collect` 를 사용합니다. `collect` 의 파라미터는 `Collector` 인데, 실무에서는 `Collectors` 클래스의 정적 메소드를 통해 필요한 `Collector` 를 얻습니다.

리스트/셋 변환은 가장 대표적인 패턴입니다. 특히 `Set` 변환은 중복 제거 특성이 있으므로 "중복 이름 제거" 같은 목적에 유용합니다.

최종 처리로 개수/합계/최댓값/최솟값도 `Collectors` 를 통해 수행할 수 있습니다. 예를 들어 `counting()` 은 `Long` 을 반환하고, `summingInt(함수)` 는 함수로 뽑은 정수 필드를 합산합니다. `maxBy/minBy` 는 `Comparator` 를 받아 `Optional<T>` 를 반환합니다.

목적	예시	결과 타입
리스트 수집	<code>collect(Collectors.toList())</code>	<code>List<T></code>
셋 수집	<code>collect(Collectors.toSet())</code>	<code>Set<T></code>
개수	<code>collect(Collectors.counting())</code>	<code>Long</code>
합계	<code>collect(Collectors.summingInt(Student::getGrade))</code>	<code>Integer</code>
최댓값	<code>collect(Collectors.maxBy(Comparator.comparing(Student::getGrade)))</code>	<code>Optional</code>

핵심 키워드

`#collect`

`#Collectors`

`#toList`

`#toSet`

`#summingInt`

12. Optional: 값 부재 처리와 orElse

`Optional` 은 값이 없을 때 발생하는 예외를 방지하기 위한 래퍼이며, `orElse` 로 기본값을 제공해 안전하게 값을 꺼낼 수 있습니다.

`Optional` 은 값이 없을 수 있는 상황에서 단순히 `get()` 을 호출하면 예외가 발생할 수 있기 때문에 도입된 타입입니다. 예를 들어 평균(`average`)은 데이터가 없으면 계산할 값이 없으므로 `OptionalDouble` 같은 타입으로 반환됩니다.

값이 있을 때는 `getAsDouble()` 로 실제 값을 얻을 수 있지만, 값이 없을 때는 예외가 발생합니다. 이때 `orElse(기본값)` 을 사용하면 값이 없을 경우 기본값으로 대체되어 예외 없이 처리됩니다.

실습에서는 `mapToDouble` 로 기본형 스트림(`DoubleStream`)을 만든 뒤 `average()` 가 `OptionalDouble` 을 반환하는 흐름을 확인했고, 데이터가 비어 있을 때 `orElse(0.0)` 로 안전하게 처리하는 패턴을 정리했습니다.

핵심 키워드

`#Optional`

`#OptionalDouble`

`#getAsDouble`

`#orElse`

`#NoSuchElementException`

13. 제네릭 와일드카드: ? super / ? extends

`? super T` 는 `T` 또는 `T`의 부모 타입만 허용하고, `? extends T` 는 `T` 또는 `T`의 자식 타입만 허용하며, 주로 메소드 파라미터에서 타입 범위를 제한할 때 사용됩니다.

스트림 API 문서나 컬렉션 API를 읽다 보면 `? super T` , `? extends T` 같은 표현이 등장합니다. 이는 제네릭 타입을 "특정 계층 범위로 제한"하기 위한 와일드카드 문법입니다.

실습에서는 `Pet` 을 부모로 `Cat` , `Dog` 를 자식으로 두고, 메소드 파라미터에서 전달 가능한 리스트 타입을 제한했습니다. 예를 들어 `List<? super Dog>` 는 `List<Dog>` 와 `List<Pet>` 는 허용하지만 `List<Cat>` 은 허용하지 않습니다.

표현	허용 범위	예시 의도
<code>? super Dog</code>	<code>Dog</code> 또는 <code>Dog</code> 의 부모	<code>Dog</code> 계열을 안전하게 받도록 제한합니다.
<code>? extends Pet</code>	<code>Pet</code> 또는 <code>Pet</code> 의 자식	<code>Pet</code> 계열(자식 포함)을 읽기 용도로 받습니다.

또한 문서에서 `Function<? super T, ? extends Stream<? extends R>>` 처럼 중첩 와일드카드가 등장할 수 있는데, 이는 "입력은 `T` 또는 부모까지 허용"하고 "반환은 `R`을 상속한 타입으로 구성된 스트림까지 허용"하는 범위 표현으로 이해하시면 됩니다.

핵심 키워드

#와일드카드

#? super

#? extends

#제네릭

#타입한정

14. MySQL 8 설치 및 Workbench 연결

MySQL Server, Workbench, 샘플을 설치하고, 포트 3306과 root 비밀번호를 설정한 뒤 Workbench에서 127.0.0.1로 테스트 연결을 성공해야 합니다.

내일 데이터베이스 수업을 위해 MySQL 8 버전을 설치했습니다. 설치 방식은 Custom 선택 후 필요한 구성요소 3가지를 설치하는 흐름입니다.

구성요소	역할	비고
MySQL Server 8.x	실제 DB 서버입니다.	서비스로 실행되어야 합니다.
MySQL Workbench	접속/쿼리 실행 툴입니다.	서버와 연동해 사용합니다.
Samples and Examples	샘플 스키마/데이터입니다.	실습에 활용될 수 있습니다.

설치 과정에서 중요한 설정은 다음과 같습니다.

- MySQL 기본 포트는 3306 이며, 접속 설정에 필요합니다.
- 인증은 패스워드 방식(권장)을 선택하고, root 비밀번호를 설정합니다.
- 설치가 끝난 뒤 “연결 성공(Connection Succeeded)”까지 확인해야 합니다.

설치 후에는 작업 관리자에서 mysqld 가 실행 중인지 확인하는 습관이 필요합니다. 서버가 죽어 있으면 Workbench에서 접속이 실패합니다.

Workbench에서 새 연결을 만들 때 입력해야 하는 값도 정리됩니다.

항목	값(예시)	의미
Hostname	127.0.0.1	로컬 PC(자기 자신)를 의미합니다.
Port	3306	MySQL 기본 포트입니다.
Username	root	관리자 계정입니다.
Password	설치 시 설정 값(예: 1234)	Store in Vault로 저장 후 테스트합니다.

기존 MySQL이 이미 설치되어 있으면 포트 충돌이 발생할 수 있으므로, 기존 설치를 유지할지 삭제 후 재설치할지 선택해야 합니다. 삭제 시에는 제어판의 프로그램 제거 후, 남아 있는 폴더(프로그램 데이터 및 사용자 AppData 로밍 등)까지 정리해야 재설치 문제가 줄어듭니다.

핵심 키워드

#MySQL8

#Workbench

#3306

#127.0.0.1

#root